

TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

3 créditos



Profesor:

Ing. Jorge Arun Zambrano Cedeño. Mg.

Titulaciones	Semestre
• <i>Tecnologías de la Información en Línea</i>	<i>Quinto</i>

NOMBRES Y APELLIDOS:

Steven Alejandro Ortiz Vite

PARALELO:

A

SEMESTRE: Quinto

ACTIVIDAD: #1

Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2026 - ENERO 2027



Actividades a desarrollar en la Unidad 1

Actividad #1

Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

La propuesta organiza el trabajo de un refugio desde el reporte de un animal hasta su atención y, cuando corresponde, su adopción. El diseño combina un modelo de dominio con reglas explícitas, creación de animales por especie y avisos automáticos a responsables. MVC separa la interacción del usuario de la lógica del sistema; Gitflow organiza la evolución de estos artefactos.

El alcance corresponde al diseño lógico solicitado en la Actividad #1 de la Unidad 1. La guía establece los actores, las especies, los estados de referencia, el uso de UML, al menos tres patrones y un repositorio público con master, develop y feature/ (Universidad Técnica de Manabí, 2026).

1 Análisis del problema

En un refugio, el reporte, el rescate, la valoración veterinaria y la asignación de responsables forman parte del mismo caso. Si sus datos se mantienen separados, puede resultar difícil identificar quién atiende al animal, qué tratamiento recibe y si está disponible para adopción. Esta situación se plantea como problema del caso académico; no describe un diagnóstico levantado en un refugio real.

El sistema centraliza un expediente por animal y permite consultar el avance. Registrar un reporte no demuestra que el rescate ya se realizó: por ello se propone el estado inicial Reportado. El responsable confirma la intervención para pasar a Rescatado. Esta distinción evita que un aviso ciudadano se interprete como un ingreso físico confirmado.

1.1 Actores y responsabilidades

Actor	Responsabilidad e interacción
Ciudadano / Reportante	Crea el caso con ubicación, descripción, contacto y categoría del animal. Consulta sus reportes y puede solicitar la adopción de un perro o gato habilitado.
Voluntario / Veterinario	Recibe avisos de nuevos rescates y consulta los casos. Como responsable asignado, confirma el rescate. El veterinario registra y finaliza tratamientos; el voluntario no emite decisiones clínicas.
Administrador del Refugio	Asigna o cambia el responsable, consulta los expedientes, habilita la adopción después de revisar la atención y aprueba o rechaza solicitudes.



1.2 Requerimientos funcionales

RF01, RF02, RF04 y RF05 desarrollan las funciones de la guía; RF10 responde a Observer. Los restantes concretan el seguimiento y la adopción. Se indican los actores y las condiciones de aceptación.

ID y función	Actor	Condición de aceptación
RF01 Crear caso de rescate	Ciudadano / Reportante	Guarda ubicación, descripción, contacto y fecha; genera un identificador. El estado inicial es Reportado y no se asigna responsable automáticamente.
RF02 Categorizar especie	Ciudadano / Reportante	El registro exige Perro, Gato o Fauna Silvestre. En fauna silvestre se describe la especie; una categoría no admitida impide completar el registro.
RF03 Consultar caso	Los tres actores	Muestra datos, responsable, estado, tratamientos y solicitudes de adopción. El ciudadano consulta los casos propios y la información necesaria para solicitar adopción.
RF04 Asignar responsable	Administrador del Refugio	Selecciona un Voluntario o Veterinario existente. El caso admite un responsable principal a la vez; una reasignación reemplaza la anterior.
RF05 Confirmar rescate	Voluntario / Veterinario asignado	Un caso Reportado pasa a Rescatado después de confirmar el ingreso del animal. La notificación inicial no realiza esta transición.
RF06 Registrar tratamiento	Veterinario	Registra descripción, fecha y veterinario. Al iniciar la atención de un animal rescatado, el caso pasa a En Tratamiento; finalizar registra la fecha de cierre.
RF07 Habilitar adopción	Administrador del Refugio	Solo un Perro o Gato rescatado, sin tratamientos activos, pasa a Adopción. La revisión del administrador es necesaria incluso si no hubo tratamiento.
RF08 Solicitar adopción	Ciudadano / Reportante	Crea una solicitud con estado Solicitada para un caso en Adopción, vinculando al solicitante. Una solicitud no equivale a una entrega aprobada.
RF09 Resolver adopción	Administrador del Refugio	Aprueba o rechaza una solicitud pendiente. Al aprobar, el caso pasa a Adoptado y no se admiten nuevas aprobaciones ni solicitudes.
RF10 Notificar nuevo rescate	Voluntario / Veterinario	Tras guardar RF01, cada responsable suscrito recibe una vez el aviso con identificador, ubicación y categoría. No se emite si el guardado falla.



1.3 Requerimientos no funcionales

Requisito	Decisión de diseño y comprobación propuesta
RNF01 Escalabilidad	Las variantes de especie se crean mediante CreadorExpediente y sus subclases. Los receptores de avisos implementan IObservadorRescate. Para agregar una variante, se revisa que los servicios no necesiten depender de la nueva clase concreta. La capacidad de atender más casos deberá medirse en una implementación.
RNF02 Mantenibilidad	Las vistas presentan datos; los controladores gestionan solicitudes; los servicios coordinan operaciones; las entidades validan estados; el repositorio persiste. Se verifica que una modificación de la interfaz no cambie reglas del dominio y que texto, UML y nombres de métodos permanezcan alineados.
RNF03 Trazabilidad de versiones	Cada avance se guarda en commits reales de una feature/ y se integra en develop. master conserva una versión revisada. El historial debe permitir identificar autor, cambio y fusión; las capturas y el enlace se incorporarán con evidencia real.

1.4 Estados y reglas del proceso

Estado	Significado y transición
Reportado	Estado derivado para un caso recibido. confirmarRescate() permite pasar a Rescatado.
Rescatado	Ingreso confirmado. Puede comenzar En Tratamiento o pasar a Adopción si es elegible y el administrador lo habilita.
En Tratamiento	Existe atención registrada. Solo se habilita Adopción cuando todos los tratamientos han finalizado y el animal es elegible.
Adopción	Perro o gato disponible para solicitudes. Una aprobación cambia a Adoptado; un rechazo mantiene Adopción.
Adoptado	Estado derivado que registra el cierre de una adopción aprobada. El expediente se conserva para consulta.

EstadoCaso contiene exactamente estos cinco valores. EstadoAdopcion contiene Solicitada, Aprobada y Rechazada. CategoriaAnimal contiene Perro, Gato y FaunaSilvestre. En el código y UML se omiten tildes en los identificadores de clases, como Adopcion; las etiquetas de estados y los nombres de los actores conservan su forma visible.

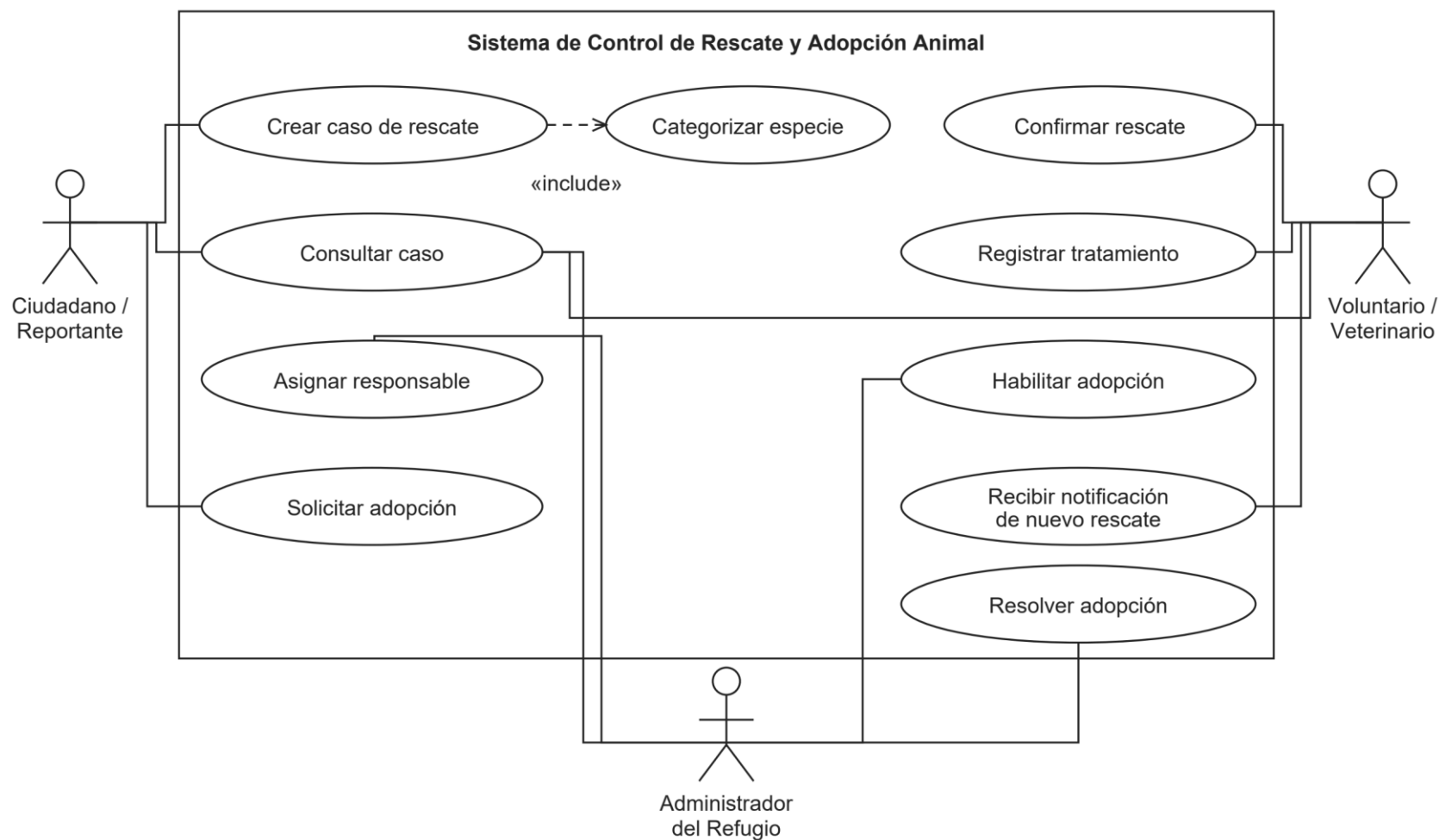


2 Diagrama UML de casos de uso

El límite del sistema incluye los objetivos de los actores. Crear caso de rescate incluye Categorizar especie porque todos los registros deben identificar la categoría del animal. Las demás interacciones son asociaciones directas: solicitar y resolver una adopción ocurren en momentos distintos, por lo que no se modelan como pasos obligatorios del registro inicial.

Registrar tratamiento se asocia al actor agrupado Voluntario / Veterinario por la forma de agrupación de la guía, con la restricción de que únicamente el rol Veterinario puede ejecutarlo. Recibir notificación de nuevo rescate expresa la recepción automática del aviso; el responsable no debe iniciar el registro para recibirlo.

Figura 1. Casos de uso del sistema



El ciudadano reporta y solicita; el responsable confirma y atiende; el administrador asigna y resuelve. «include» indica la categorización obligatoria dentro del registro.



2.1 Condiciones de los casos principales

Crear caso de rescate exige datos completos y una categoría admitida. El resultado es un CasoRescate persistido en Reportado. Si falta un dato o falla la persistencia, se informa el error y no se anuncia un caso que el sistema no haya guardado.

Asignar responsable exige que el caso y el responsable existan. El administrador decide la asignación; Observer solamente difunde el aviso. Confirmar rescate exige que la persona sea el responsable asignado y que el estado anterior sea Reportado.

Habilitar adopción exige elegibilidad de especie y ausencia de tratamiento activo. Resolver adopción exige una solicitud pendiente y que no exista otra aprobada para el caso. Las solicitudes rechazadas se conservan; al aprobar una se cierran como rechazadas las demás pendientes para evitar decisiones posteriores contradictorias.

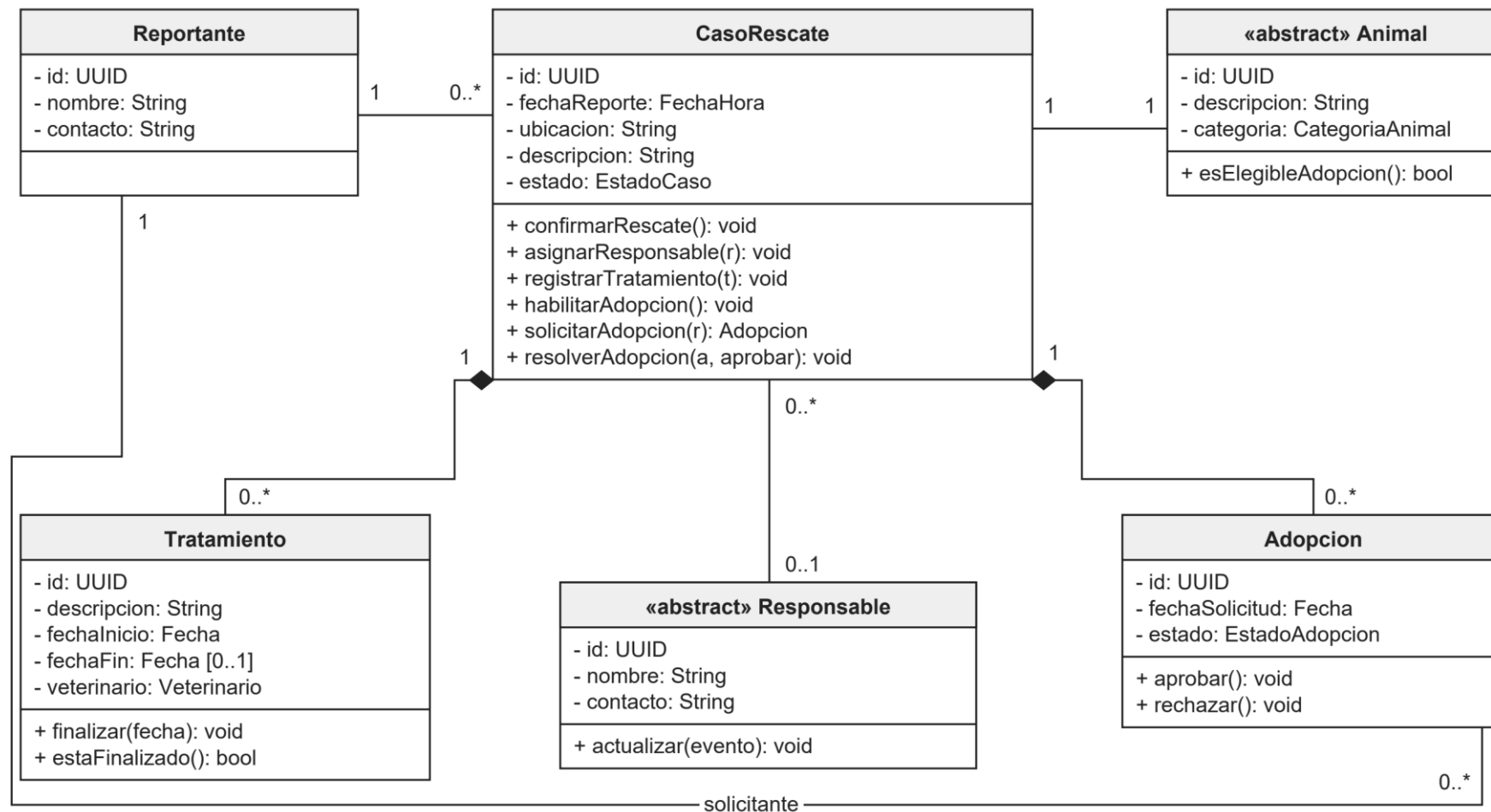
3 Diagrama de clases UML

El modelo se presenta en cuatro vistas del mismo diagrama de clases para conservar la legibilidad. Cada vista muestra los miembros relevantes; las clases repetidas se refieren al mismo tipo. La vista A expone el dominio; B muestra Factory Method; C muestra Observer; D conecta MVC con servicios y persistencia. Las firmas abreviadas usan r para Reportante o Responsable según el método, t para Tratamiento, a para Adopcion y e para EventoNuevoRescate.

CasoRescate es la raíz del expediente: conserva su identidad, el estado y las reglas de transición. Un caso se relaciona con un único Animal; un Reportante puede originar varios casos. Los Tratamiento y las solicitudes Adopcion pertenecen a un solo caso, por lo que se usa composición. Animal y Responsable se asocian sin composición porque representan identidades del dominio que no se eliminan al cerrar el expediente.



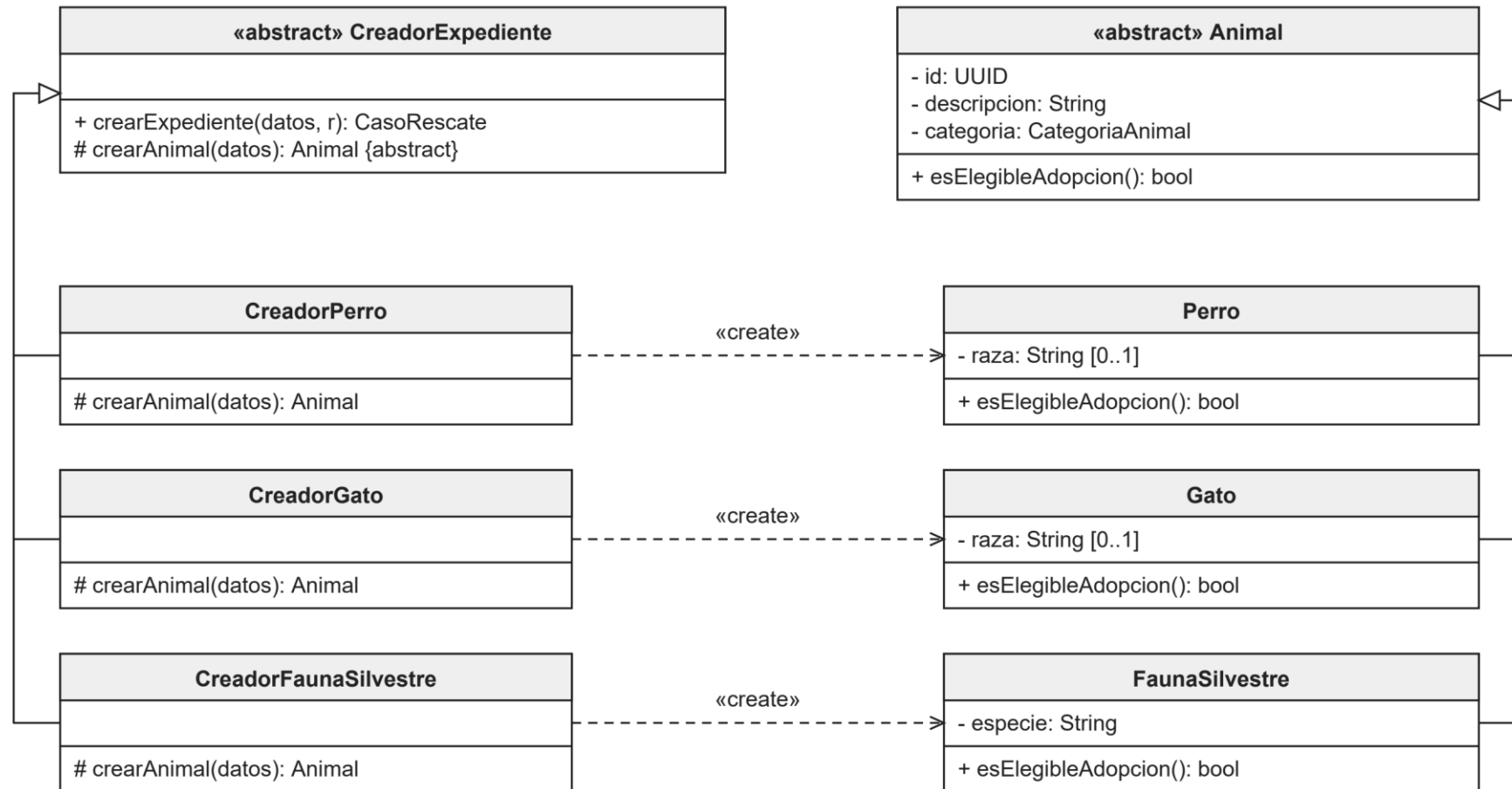
Figura 2. Diagrama de clases UML vista A del dominio



Multiplicidades: un reportante por caso; un animal por caso; cero o un responsable principal; cero o más tratamientos y solicitudes. Cada Tratamiento registra un Veterinario. Los rombos sólidos representan la pertenencia al expediente.



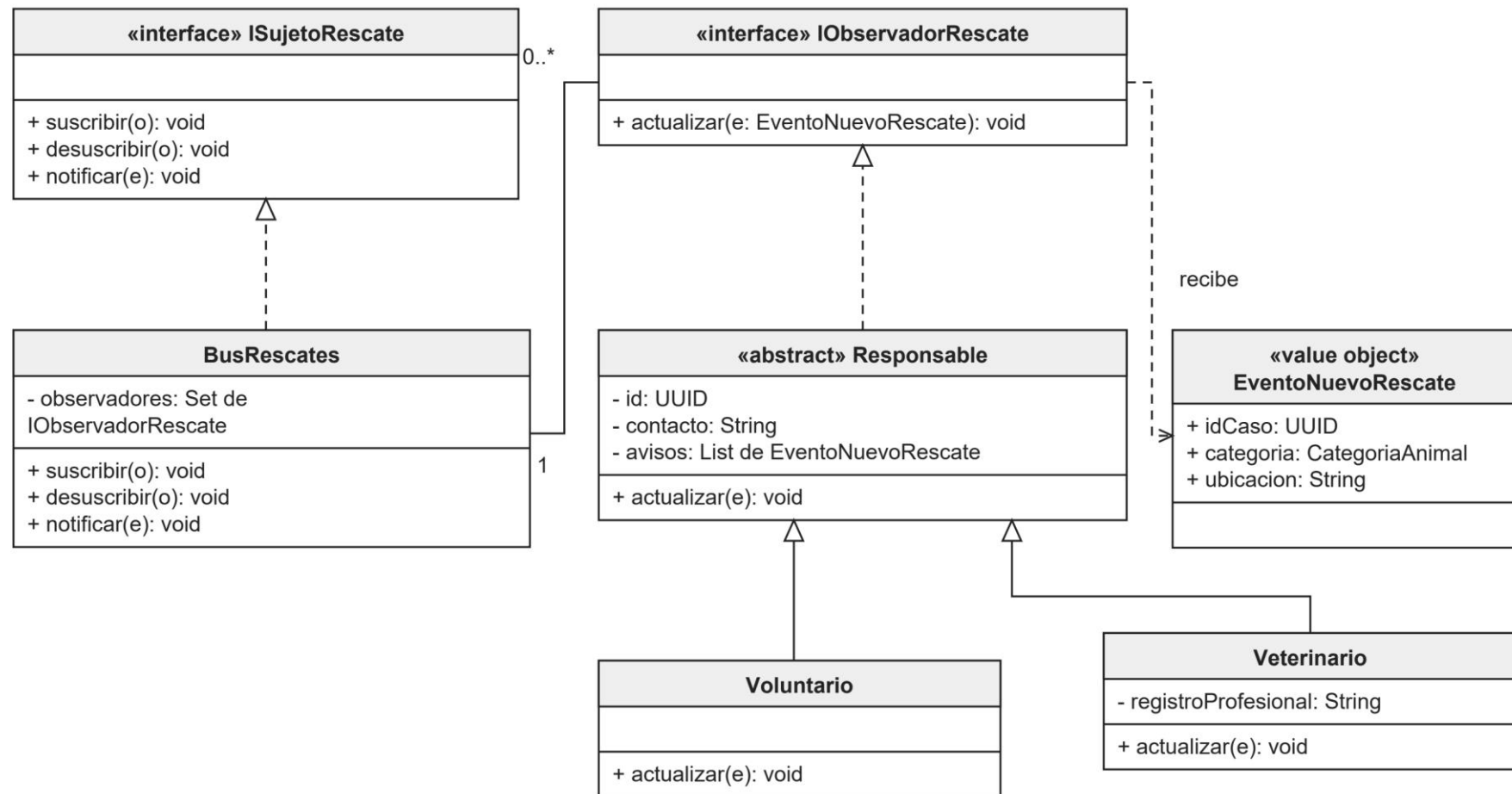
Figura 3. Diagrama de clases UML vista B de Factory Method



CreadorExpediente define crearExpediente() y declara crearAnimal() como método fábrica protegido. Cada creador concreto devuelve un Animal de su categoría; las flechas de triángulo vacío representan herencia.



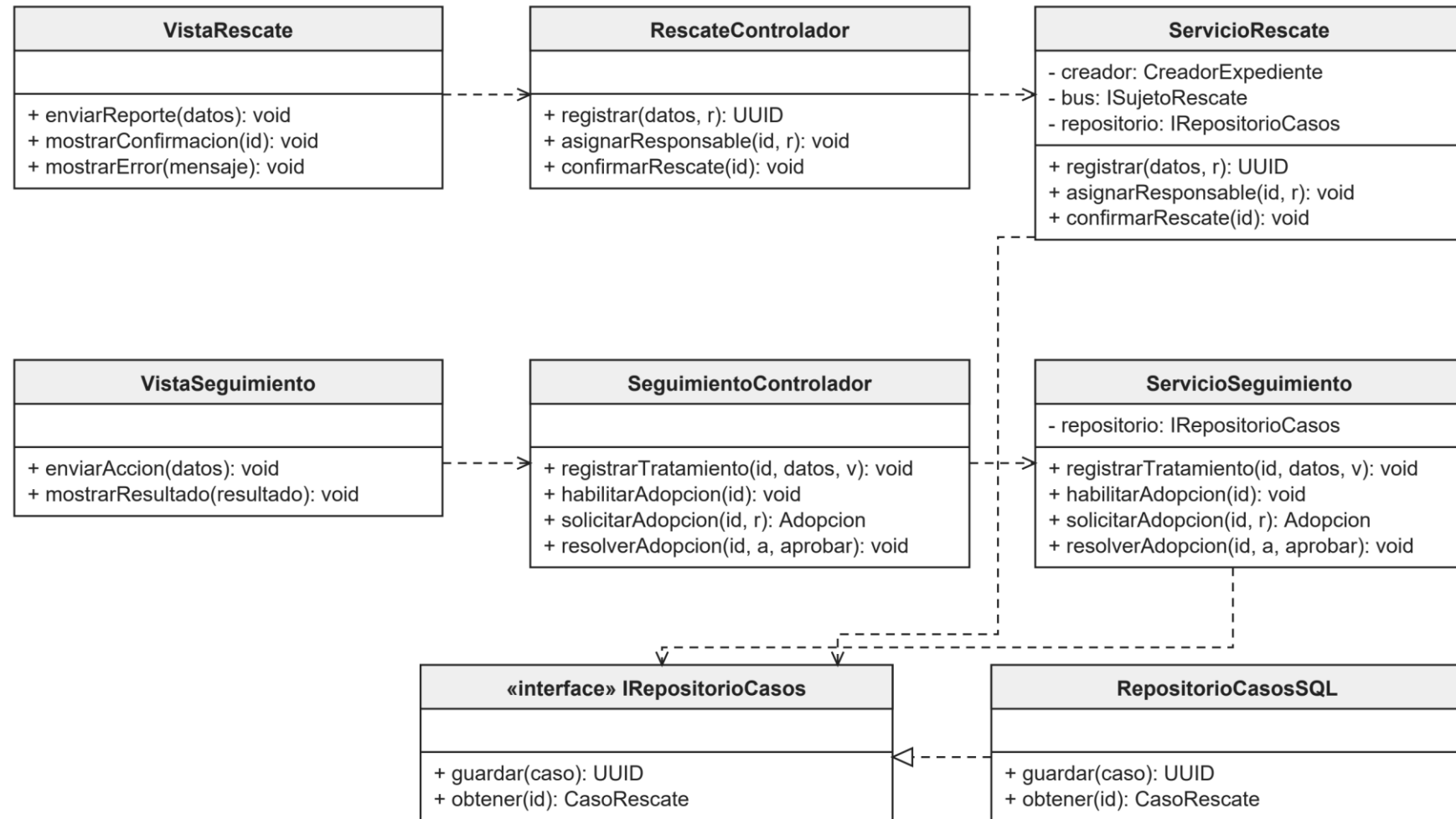
Figura 4. Diagrama de clases UML vista C de Observer



BusRescates realiza ISujetoRescate y conserva suscriptores IObservadorRescate. Responsable realiza el contrato de recepción; Voluntario y Veterinario son especializaciones. Las realizaciones usan línea discontinua y triángulo vacío.



Figura 5. Diagrama de clases UML vista D de MVC y persistencia



Las vistas dependen de controladores, y estos de servicios. Los atributos de ServicioRescate remiten a CreadorExpediente e ISujetoRescate, definidos en B y C. Ambos servicios usan IRepositorioCasos; RepositorioCasosSQL realiza ese contrato. El dominio se define en A.



3.1 Responsabilidades e invariantes

ServicioRescate coordina registro, asignación y confirmación. ServicioSeguimiento coordina tratamientos y adopciones. Ambos recuperan o guardan la raíz CasoRescate mediante IRepositorioCasos. Los controladores comprueban el rol del usuario y delegan; el dominio comprueba las reglas del estado. Estas validaciones también deben aplicarse si una operación se invoca fuera de la interfaz.

Cada Tratamiento referencia exactamente a un Veterinario, aunque el responsable principal del caso sea un Voluntario. El atributo veterinario de la vista A expresa esta relación tipada con la especialización de la vista C. La asignación de un responsable principal no limita al caso a un único tratamiento.

La colección de solicitudes admite varios registros históricos, pero como máximo uno Aprobado. El vínculo de Adopcion con Reportante identifica al solicitante, que puede ser una persona distinta del reportante original del rescate. El expediente conserva ambos vínculos para no confundir quién reportó con quién adoptó.

Perro y Gato devuelven verdadero en esElegibleAdopcion(); FaunaSilvestre devuelve falso. Este resultado representa elegibilidad por categoría, no aprobación clínica ni administrativa. La fauna silvestre permanece en el circuito de registro y atención; su destino posterior queda fuera del alcance definido para esta solución. No se formula aquí un procedimiento legal de manejo de fauna.

El Administrador del Refugio se modela como rol de autorización y no como entidad con comportamiento propio. Se evita una clase Administrador sin responsabilidad de dominio. Las notificaciones se representan con el valor EventoNuevoRescate y la bandeja avisos del Responsable; no se añade un canal de correo o mensajería que la guía no exige.

4 Aplicación de patrones de diseño

Se utilizan tres patrones con funciones diferentes: Observer difunde avisos, Factory Method crea las variantes del animal y MVC organiza la interacción con el modelo. La selección responde a los problemas del caso y a las sugerencias docentes; no se emplea Singleton porque no se ha identificado un recurso que deba imponer una instancia global única.

4.1 Observer para nuevos rescates

Problema que resuelve. El registro de un caso debe avisar a voluntarios y veterinarios sin incorporar en ServicioRescate una lista de clases receptoras concretas. La estructura sujeto-observador permite mantener un conjunto de receptores y actualizarlos a través de un contrato común (Schmidt, s. f.-a).

Aplicación y funcionamiento. BusRescates implementa ISujetoRescate con suscribir(), desuscribir() y notificar(). Responsable implementa IObservadorRescate mediante actualizar(e), operación que registra el aviso en su bandeja. Voluntario y Veterinario heredan este contrato. Los responsables disponibles se suscriben al iniciar su participación; la suscripción termina cuando dejan de recibir avisos.

Después de guardar un nuevo CasoRescate, ServicioRescate construye EventoNuevoRescate con idCaso, categoria y ubicacion, y llama a notificar(e). BusRescates recorre el conjunto de suscriptores y ejecuta actualizar(e) una vez por receptor. Un Set evita suscripciones duplicadas. El envío no decide quién atenderá el caso: esa decisión corresponde al administrador.

Justificación técnica y evidencia UML. La vista C muestra los contratos y sus realizaciones; la secuencia de la figura 8 expone la colaboración dinámica. Agregar un receptor compatible no modifica el registro del caso. El desacoplamiento mejora la mantenibilidad y la reutilización del aviso. La ejecución propuesta es síncrona; el patrón por sí solo no garantiza entregas durables ni procesamiento distribuido.

Límite operativo. Un fallo de recepción no debe invalidar un caso que ya se guardó. BusRescates debe aislar el fallo de cada suscriptor y continuar con los demás; el caso no se registra por segunda vez para reintentar un aviso. La persistencia de avisos y una política automática de reintentos requerirían un alcance adicional y no se presuponen en este diseño.

4.2 Factory Method para crear el expediente por especie

Problema que resuelve. Si el controlador construye directamente Perro, Gato y FaunaSilvestre, conoce los detalles de cada categoría y cambia al extender la creación. Factory Method separa la creación del uso del producto y permite que una subclase concrete el objeto que se devuelve (Schmidt, s. f.-b).

Aplicación y funcionamiento. CreadorExpediente es el creador abstracto. Su operación crearExpediente(datos, r) valida los datos comunes, invoca crearAnimal(datos) y construye CasoRescate con estado Reportado. crearAnimal() es el método fábrica protegido y abstracto. CreadorPerro, CreadorGato y CreadorFaunaSilvestre lo sobrescriben para devolver Perro, Gato y FaunaSilvestre, respectivamente, a través del tipo Animal.

La configuración de la aplicación contiene una correspondencia entre CategoriaAnimal y el creador apropiado. Antes de ejecutar el registro, esta configuración suministra el CreadorExpediente concreto a ServicioRescate. La selección de categoría queda en ese punto de composición, no repetida en los controladores ni en el método común de creación.

Justificación técnica y evidencia UML. La vista B representa las dos jerarquías y las dependencias de creación. La figura 7 muestra el caso Perro; las otras categorías cumplen el mismo contrato. El método común conserva las reglas compartidas y los creadores encapsulan los detalles variables. Una nueva categoría requiere añadir su producto y creador, y actualizar la configuración; el servicio continúa usando la abstracción. Esta es una extensión del diseño, no una función adicional exigida.

4.3 MVC para organizar la interacción

Problema que resuelve. Una pantalla que guarda datos, controla permisos y decide estados concentra cambios de distintas causas. MVC separa la presentación, la atención de solicitudes y el estado con sus reglas; esto permite desarrollar el modelo con independencia de la interfaz (Smith, 2026).

Aplicación y funcionamiento. VistaRescate presenta el reporte y muestra confirmaciones o errores; RescateControlador recibe registrar(), asignarResponsable() y confirmarRescate(). VistaSeguimiento y SeguimientoControlador atienden las operaciones de tratamiento y adopción. El modelo comprende ServicioRescate, ServicioSeguimiento, las entidades, los creadores, las notificaciones y los contratos de persistencia.

Los controladores delegan la operación solicitada a los servicios; no construyen animales concretos ni manipulan consultas SQL. Los servicios coordinan el caso y las entidades protegen sus invariantes. Las vistas muestran resultados sin modificar directamente el estado del expediente. No se exige una tecnología específica de interfaz o un framework para aplicar esta separación.

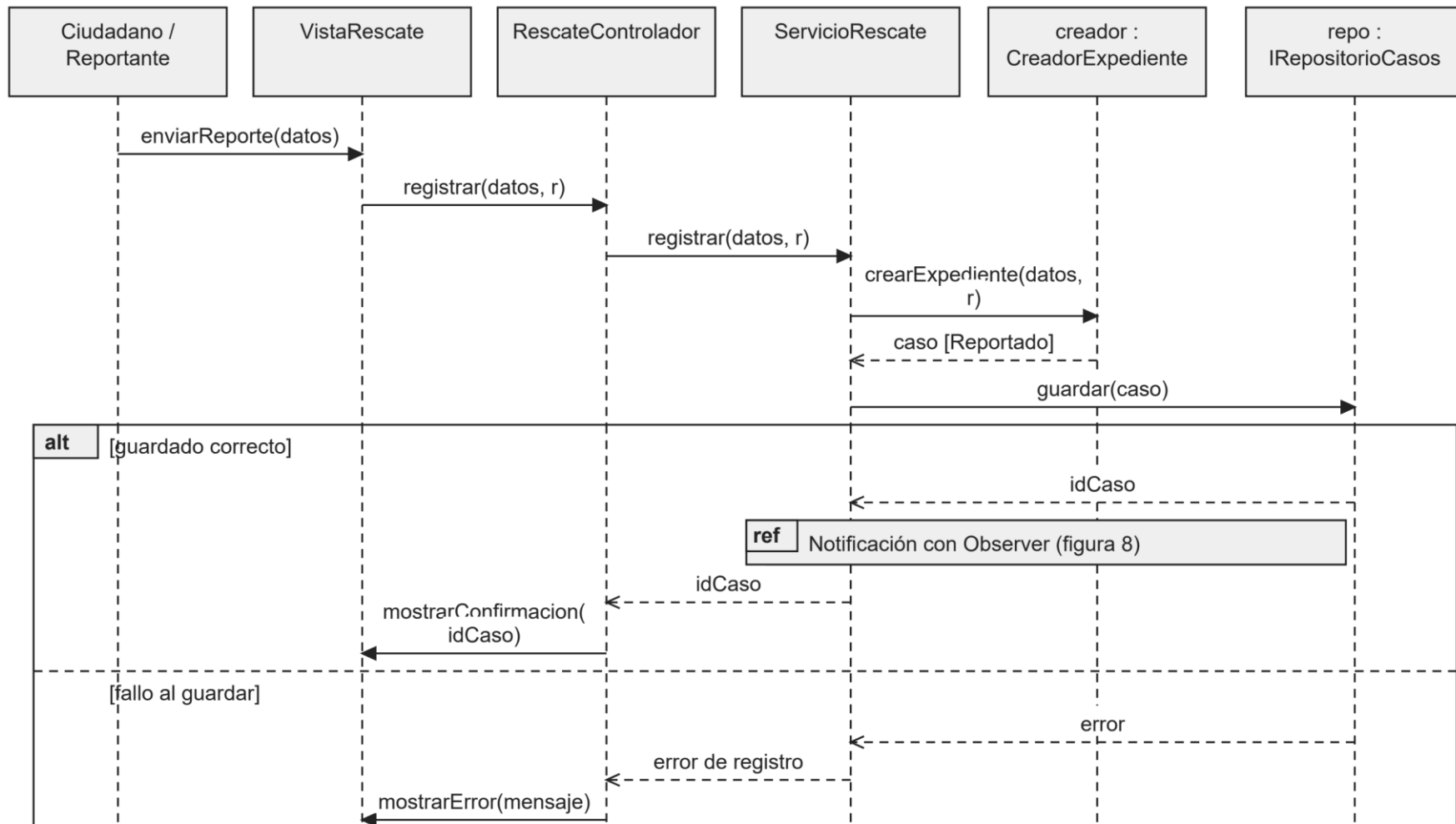
Justificación técnica y evidencia UML. La vista D y la figura 6 muestran la cadena vista–controlador–modelo. La figura 9 sitúa estos componentes dentro de la arquitectura. La separación permite cambiar formularios, persistencia o reglas con menor propagación de modificaciones. Se reutiliza el modelo en otra presentación y se evitan controladores que concentren decisiones de negocio.

5 Diagramas de secuencia UML

El escenario Registrar nuevo rescate y notificar a voluntarios se descompone en secuencias complementarias para que los mensajes puedan leerse. La figura 6 muestra la interacción MVC y el guardado; la figura 7 detalla la creación del objeto; la figura 8 continúa con los avisos después de un guardado exitoso. Todas usan las operaciones definidas en las vistas del diagrama de clases.



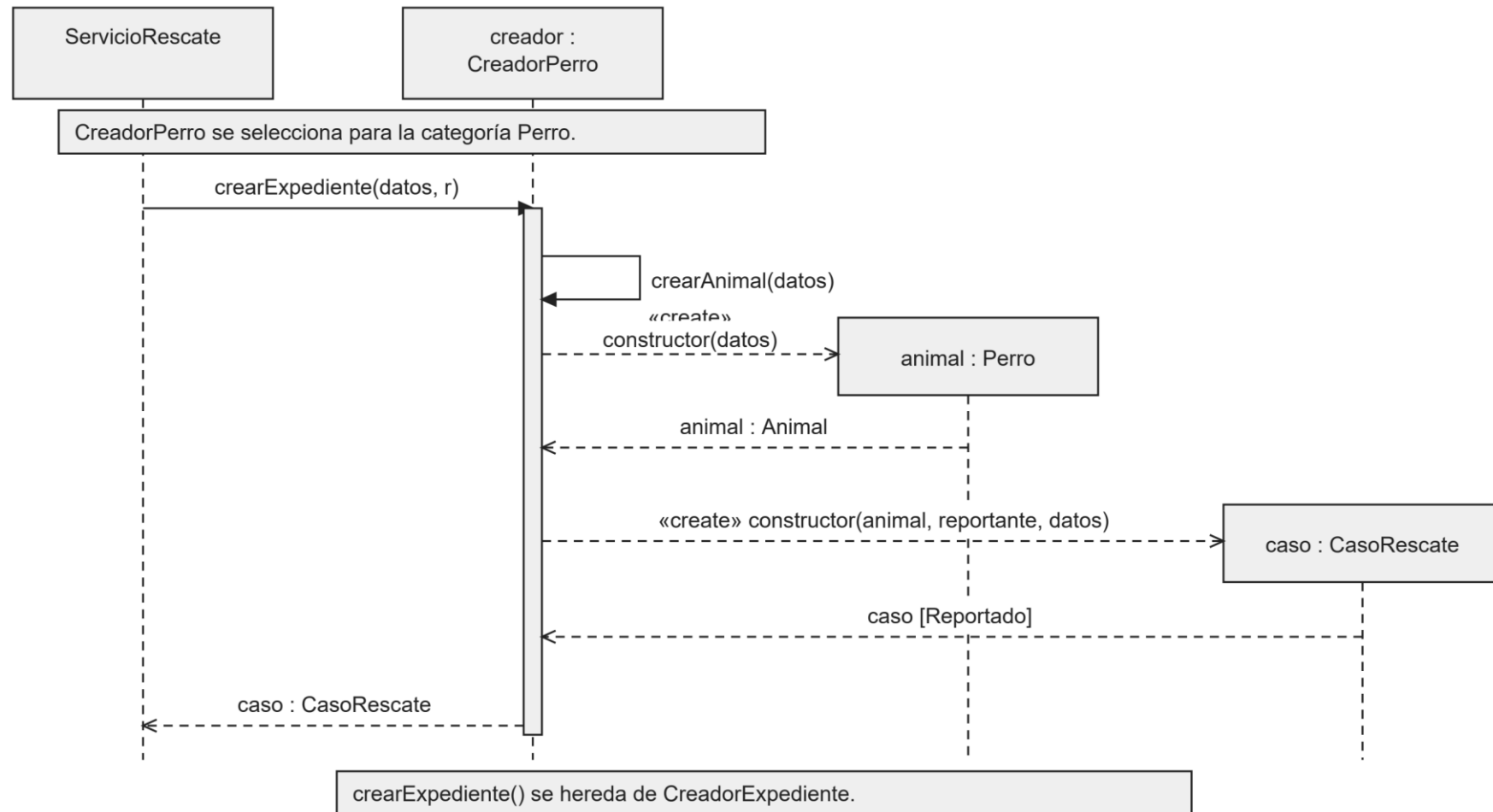
Figura 6. Registro del rescate mediante MVC



ServicioRescate obtiene el expediente y lo guarda. La alternativa distingue éxito y error de persistencia. El aviso de la figura 8 se ejecuta después del guardado correcto y antes de devolver la confirmación.



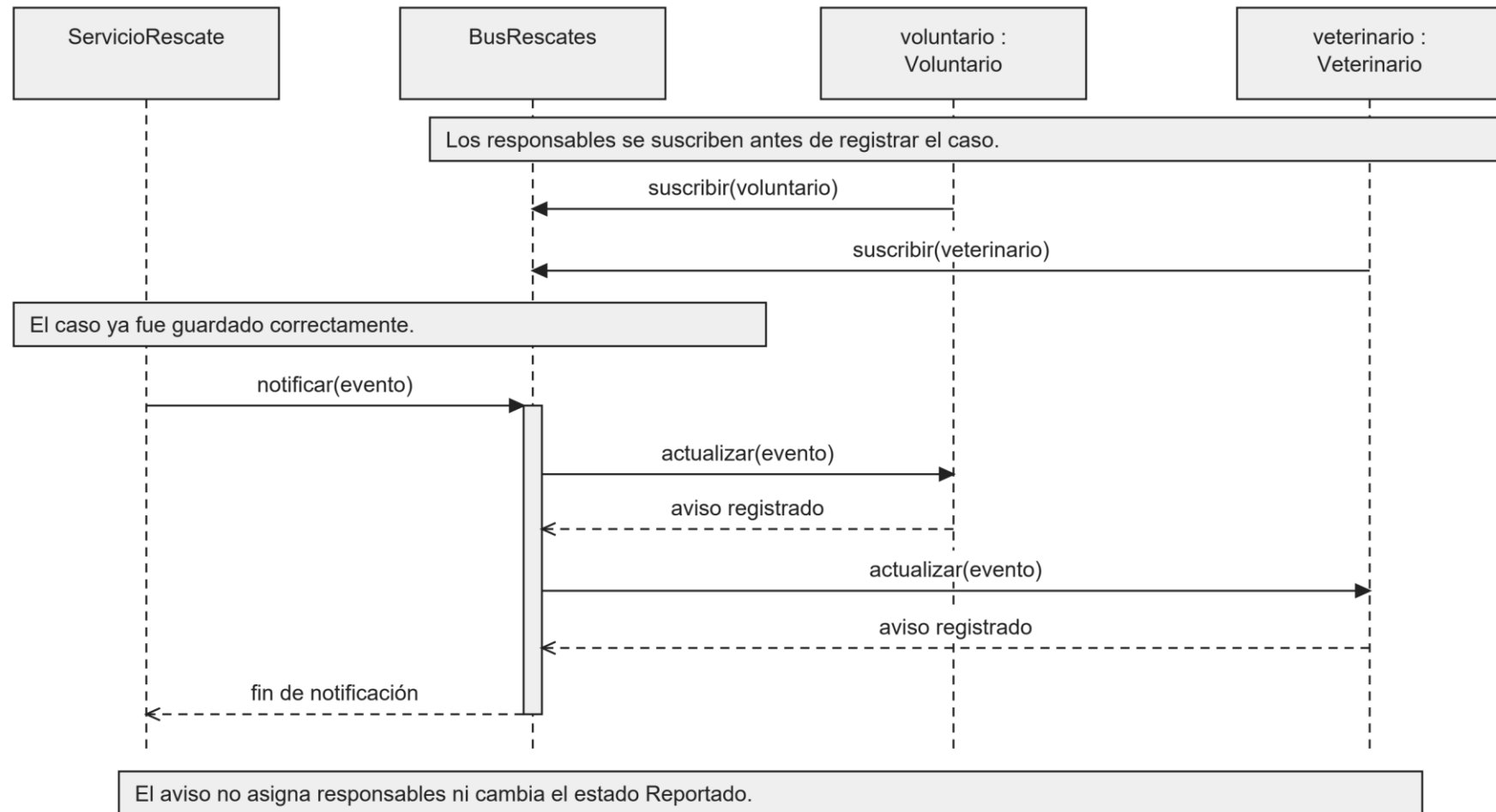
Figura 7. Creación del expediente de un perro con Factory Method



crearExpediente() se hereda de CreadorExpediente y llama al crearAnimal() sobrescrito por CreadorPerro. Los mensajes constructor representan creación de instancias, implícita en las clases UML.



Figura 8. Notificación de nuevo rescate con Observer



El ejemplo presenta dos suscriptores, un Voluntario y un Veterinario. Cada uno recibe actualizar(evento) una vez. En general, BusRescates recorre todos los IObservadorRescate suscritos.



6 Arquitectura lógica general

La solución se organiza como una aplicación modular con presentación MVC y un modelo dividido por responsabilidades. El flujo ordinario va de la vista al controlador, del controlador al servicio y del servicio al dominio y a sus contratos auxiliares. Esta organización describe dependencias del software, sin proponer servidores físicos ni una infraestructura distribuida.

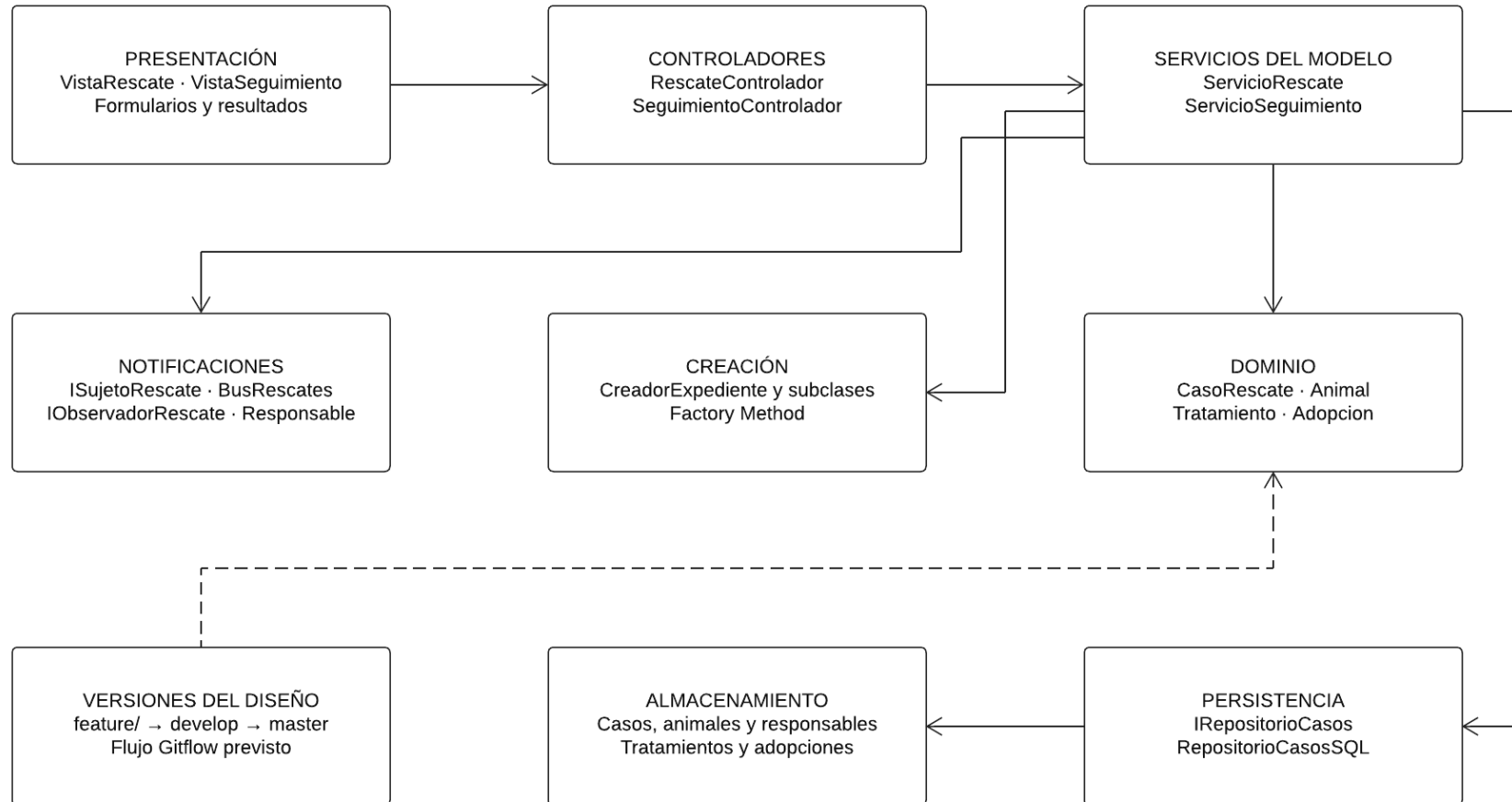
`IRepositorioCasos` ofrece `guardar(caso)` y `obtener(id)`. `RepositorioCasosSQL` implementa ambas operaciones y conserva, mediante el expediente, los vínculos con el animal, el reportante, el responsable, los tratamientos y las adopciones. La escritura del caso y sus cambios relacionados debe ser consistente; una adopción aprobada no puede quedar almacenada sin el cambio de estado del caso.

El punto de composición conecta las implementaciones con los servicios: un `RepositorioCasosSQL` para `IRepositorioCasos`, `BusRescates` para `ISujetoRescate` y el creador concreto según `CategoriaAnimal`. No forma parte de la lógica de atención al ciudadano; configura las dependencias al iniciar la aplicación o al preparar la operación.

Esta estructura favorece la mantenibilidad al delimitar qué componente cambia por una nueva pantalla, una regla de transición o un mecanismo de almacenamiento. También prepara la extensión de especies y receptores sin duplicar el flujo principal. La escalabilidad de carga es una cualidad que deberá validarse con datos y pruebas de una implementación, no un resultado demostrado por los diagramas.



Figura 9. Arquitectura lógica y relación con Gitflow



Las flechas continuas representan colaboración entre componentes. La flecha discontinua desde VERSIONES DEL DISEÑO indica el versionado de los artefactos; Gitflow no es un componente de ejecución del sistema.

7 Aplicación concreta de SOLID

El control de las dependencias permite limitar la propagación de cambios. El diseño utiliza abstracciones y responsabilidades separadas para procurar este objetivo; su cumplimiento deberá comprobarse también al implementar los contratos (Martin, 2000).

Principio	Decisión aplicada al sistema
S Responsabilidad única	VistaRescate presenta; RescateControlador recibe solicitudes; ServicioRescate coordina el registro; CasoRescate valida transiciones; RepositorioCasosSQL persiste; BusRescates difunde avisos. Cambiar un formulario no obliga a modificar el estado del dominio.
O Abierto y cerrado	La creación se amplía con subclases de CreadorExpediente y Animal. Los avisos admiten nuevas implementaciones de IObservadorRescate. Los servicios usan estos contratos; únicamente cambia la configuración que conecta las variantes.
L Sustitución de Liskov	Perro, Gato y FaunaSilvestre pueden utilizarse donde se espera Animal. esElegibleAdopcion() siempre devuelve bool; devolver falso para fauna silvestre respeta el contrato. Animal no promete que todos sus subtipos sean adoptables. Los creadores siempre entregan un CasoRescate válido.
I Segregación de interfaces	IObservadorRescate solo declara actualizar(e). ISujetoRescate concentra suscripción y aviso. IRepositorioCasos contiene guardar y obtener. Un receptor no debe implementar persistencia, y un repositorio no necesita operaciones de notificación.
D Inversión de dependencias	ServicioRescate depende de CreadorExpediente, ISujetoRescate e IRepositorioCasos; ServicioSeguimiento depende de IRepositorioCasos y del dominio. Las implementaciones concretas se suministran desde la configuración. El servicio no abre directamente una conexión SQL.

Para conservar estos principios, se revisaría que crearAnimal() no retorne una categoría distinta de la configurada, que un observador no modifique el CasoRescate recibido indirectamente por su identificador y que el repositorio respete el mismo comportamiento de guardado al sustituir su implementación. Estas son condiciones de verificación propuestas, no resultados de pruebas ejecutadas.



8 Gitflow y trazabilidad del diseño

Gitflow diferencia una línea estable y una línea de integración, y permite trabajar funcionalidades por separado. El modelo original describe master y develop con responsabilidades distintas y la fusión de funcionalidades hacia develop (Driessen, 2010).

Para esta actividad se adopta el conjunto de ramas solicitado por el docente: master, develop y feature/. master conservará la versión estable revisada del diseño; develop reunirá avances compatibles; cada feature/ contendrá un cambio delimitado, como requerimientos, casos de uso o patrones. La guía no exige implementar las ramas adicionales release/ y hotfix/ del modelo completo, por lo que no se amplía la práctica con ellas.

Rama prevista	Contenido y criterio de integración
master	Documento, diagramas y especificaciones coherentes de una versión revisada. Se actualiza después de verificar el trabajo integrado en develop.
develop	Integra los avances de las feature/ y permite corregir discrepancias entre texto, clases y secuencias antes de consolidar una versión estable.
feature/analisis-requerimientos	Problema, actores, RF, RNF y reglas del estado. Nace de develop y se fusiona cuando estos elementos sean consistentes.
feature/uml-casos-uso	Casos de uso y sus relaciones con los actores y RF.
feature/uml-clases	Vistas del modelo, responsabilidades, multiplicidades e invariantes.
feature/patrones-diseno	Justificación de Observer, Factory Method y MVC; secuencias compatibles con las clases.

Flujo previsto. Se parte de develop, se crea una feature/, se realizan cambios verificables y se guardan commits con mensajes concretos. Tras revisar el avance, se fusiona hacia develop conservando un commit de integración. Cuando requisitos, figuras y texto coincidan, develop se integra en master. Este recorrido permite relacionar una decisión del diseño con los cambios que la incorporaron.

La trazabilidad de versiones es diferente del historial clínico del animal: Git documenta la evolución de los archivos del proyecto y no sustituye el almacenamiento de casos del sistema. Se versionarán especificaciones y exportaciones de los diagramas. El código se incorporará si se implementa después; la actividad actual se concentra en el diseño lógico.

Las ramas anteriores expresan una propuesta. El estudiante incorporará el repositorio público y las capturas reales en la sección 10; este documento no acredita ejecuciones de Git. La figura 9 relaciona el flujo con los artefactos del diseño.



9 Conclusiones

La arquitectura reúne la información de rescate en un expediente con transiciones controladas y responsables identificados. La diferencia entre Reportado y Rescatado impide confirmar un ingreso a partir de una notificación; las condiciones de tratamiento y elegibilidad evitan habilitar adopciones incompatibles con el modelo.

Observer desacopla los avisos del registro, Factory Method concentra la creación por especie y MVC distribuye la interacción y la lógica. Las interfaces de persistencia y notificación apoyan la extensión del sistema y la revisión de sus dependencias. Gitflow permitirá mantener una versión estable y reconstruir los avances del diseño mediante evidencia real.

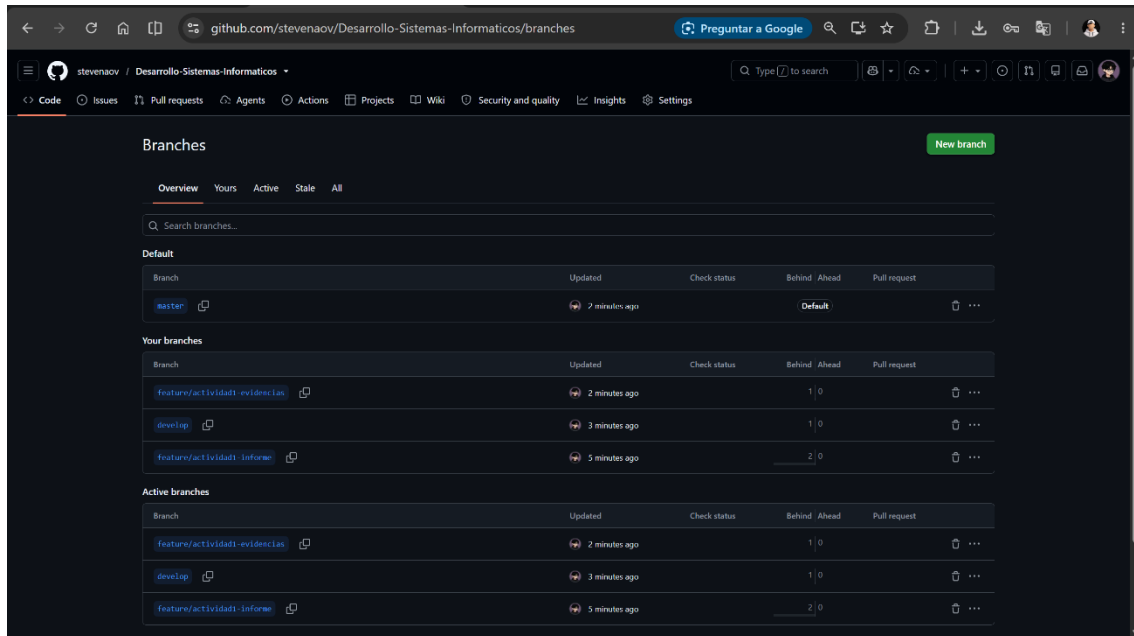


10 Evidencias

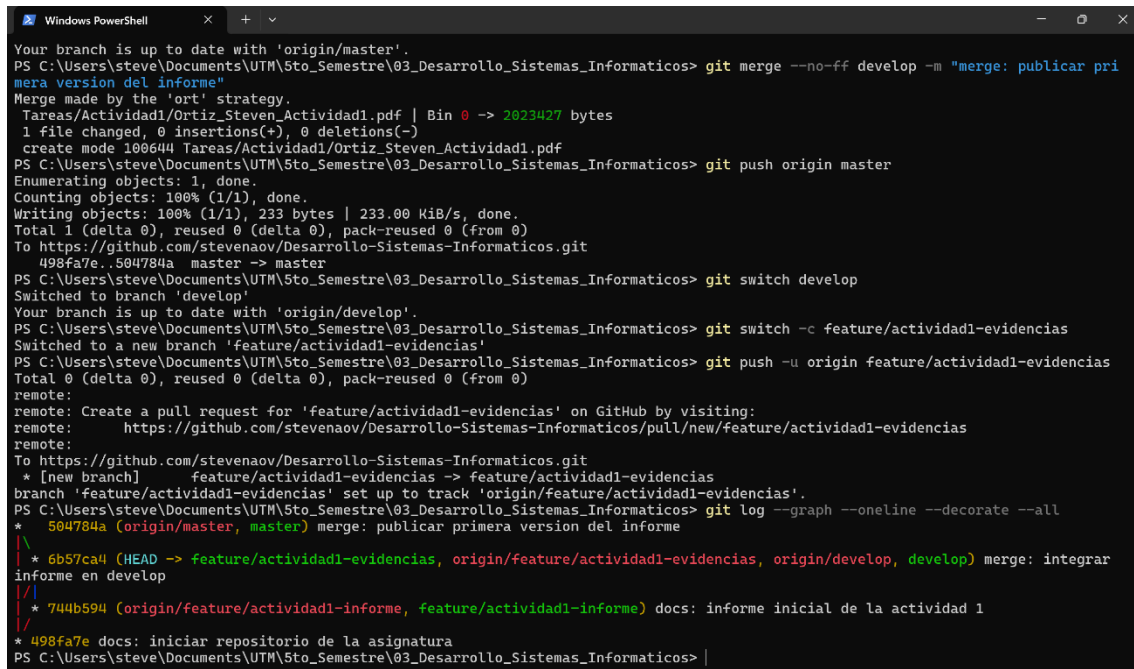
Enlace público del repositorio GitHub

<https://github.com/stevenaov/Desarrollo-Sistemas-Informaticos.git>

Evidencia de ramas Gitflow



Evidencia del historial de commits





11 Bibliografía

Driessen, V. (2010). *A successful Git branching model*. nvie.com. <https://nvie.com/posts/a-successful-git-branching-model/>

Martin, R. C. (2000). *Design principles and design patterns*. Object Mentor. https://objectmentor.com/resources/articles/Principles_and_Patterns.pdf

Schmidt, D. C. (s. f.-a). *Introduction to patterns and frameworks*. Vanderbilt University. <https://www.cs.wm.edu/~dcschmidt/PDF/patterns-intro.pdf>

Schmidt, D. C. (s. f.-b). *The Factory Method pattern motivating example*. <https://www.cs.wm.edu/~dcschmidt/cs251/2021-PDFs/factory-method-pt1.pdf>

Smith, S. (2026). *Overview of ASP.NET Core MVC*. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/mvc/overview?view=aspnetcore-10.0>

Universidad Técnica de Manabí. (2026). *Guía de actividades de la Unidad 1 de Desarrollo de Sistemas Informáticos*.